🎤 HONEST INTERVIEW ANSWER (MEMORIZE)

“It may look simple on the surface, but the project focuses on file processing, binary data handling, and full-stack integration, which are common real-world challenges.”

🧠 HOW TO EXPLAIN API WORKING (STRUCTURED WAY)

Use this **4-step flow** for **any API** in your project.

🔁 GENERAL API FLOW (SAY THIS FIRST)

“The frontend sends files using multipart/form-data, the backend processes them in memory using streams, and returns the result as a binary response which the frontend downloads using Blob.”

That one sentence already sounds **professional**.

🔍 EXPLAIN ONE API IN DETAIL (EXAMPLE: PDF MERGE)

1️⃣ Frontend

```
const formData = new FormData();

formData.append("files", file1);

formData.append("files", file2);


fetch("/api/files/merge-pdf", {

  method: "POST",

  body: formData

});
```

📌 **Explain like this**:

“The frontend collects multiple PDF files and sends them as multipart form data.”

2️⃣ Controller

```
@PostMapping("/merge-pdf")

public ResponseEntity<byte[]> mergePdf(@RequestParam("files") MultipartFile[] files)
```

📌 Say:

“Spring Boot automatically maps uploaded files to a MultipartFile array.”

3️⃣ Processing

PDFMergerUtility merger = new PDFMergerUtility();

ByteArrayOutputStream outputStream = new ByteArrayOutputStream();

📌 Say:

"Files are merged in memory using PDFBox and written to a byte stream."

4️⃣ Response

return new ResponseEntity<>(byte[], headers, HttpStatus.OK);

📌 Say:

"The merged PDF is returned as a binary response without saving it on the server."

5️⃣ Frontend Download

const blob = await response.blob();

📌 Say:

"The frontend converts the response to a Blob and triggers a file download."

🔄 APPLY SAME LOGIC TO OTHER APIs

Image Compression

- Upload image
- Compress using Thumbnailator
- Return compressed image as byte[]

PDF → Word

- Upload PDF
- Extract text using PDFBox
- Generate Word using Apache POI
- Return .docx as binary

👉 Same flow, different processing logic.

🎤 INTERVIEW-READY SHORT EXPLANATION (MEMORIZE)

"All APIs follow a consistent pattern: multipart upload, in-memory processing, and binary file response, which keeps the application fast and stateless."

🧠 WHY THIS MAKES YOUR PROJECT STRONG

Most students:
❌ Just show UI
❌ Don't understand file flow

You:
✓ Understand request–response lifecycle
✓ Know multipart handling
✓ Handle binary data
✓ Avoid disk I/O

That's **real backend knowledge**.

Prepare **API flow diagram**
2️⃣ Write **README API section**
3️⃣ Practice **mock interview Q&A**

1️⃣ API FLOW DIAGRAM (EASY TO EXPLAIN)

You can **draw this on paper / whiteboard / README**.

```
[ React UI ]
    |
    | (multipart/form-data)
    | POST /api/files/{action}
    v
[ Spring Boot Controller ]
    |
    | MultipartFile / MultipartFile[]
    v
[ In-Memory Processing ]
(PDFBox / Thumbnailator / POI)
    |
    | byte[]
    v
[ HTTP Binary Response ]
```

(Content-Disposition: attachment)

|

v

[Frontend Blob Download]

One-line explanation (MEMORIZE):

"The frontend uploads files as multipart data, the backend processes them in memory, and returns the result as a downloadable binary response."

2️⃣ README — API SECTION (COPY-PASTE READY)

🔌 API Endpoints

📄 Merge PDF Files

Endpoint

POST /api/files/merge-pdf

Request

- Content-Type: multipart/form-data
- Parameter: files (multiple PDF files)

Response

- Merged PDF file (application/pdf)

🖼️ Compress Image

Endpoint

POST /api/files/compress-image

Request

- Content-Type: multipart/form-data
- Parameters:
  - file (image)
  - quality (compression level)

Response

- Compressed image file

📝 Convert PDF to Word

Endpoint

POST /api/files/pdf-to-word

Request

- Content-Type: multipart/form-data
- Parameter: file (PDF file)

Response

- Word document (.docx)

🔁 Common Processing Pattern

- Multipart upload
- In-memory processing (no disk storage)
- Binary response (byte[])
- Frontend download using Blob

3️⃣ MOCK INTERVIEW Q&A (VERY IMPORTANT)

Q1️⃣ How does file upload work in your project?

Answer:

"The frontend sends files using multipart/form-data, which Spring Boot maps to MultipartFile objects for processing."

Q2️⃣ Why did you use byte[] instead of saving files on the server?

Answer:

"Processing files in memory improves performance, avoids disk I/O, and keeps the application stateless."

Q3️⃣ How do you handle file downloads on the frontend?

Answer:

"The backend sends a binary response, which is converted into a Blob on the frontend to trigger file download."

Q4️⃣ What libraries did you use and why?

Answer:

"PDFBox for PDF processing, Thumbnailator for image compression, and Apache POI for Word document generation."

Q 5️⃣ What are the limitations of PDF to Word conversion?

Answer:

"Text extraction works well, but complex layouts and images may not be preserved accurately."

Q 6️⃣ How is CORS handled?

Answer:

"CORS is configured at the controller level to allow frontend requests from a specific origin."

Q 7️⃣ How would you scale this application?

Answer:

"By adding a service layer, validating file sizes, and deploying behind a load balancer since the app is stateless."

🧠 **FINAL INTERVIEW POWER LINE (USE THIS)**

"This project demonstrates real-world file handling, multipart APIs, in-memory processing, and full-stack integration."